Guía Semana 10 — IA Generativa Multimodal y Edge AI

Nov 9, 2026 · @AntonioJGL

Programación Móvil Avanzada · Universidad de La Guajira. Esta semana tu app habla con un modelo de IA sin exponer nunca la API key.

Qué vas a aprender y qué traer

Al final de la semana tendrás un chat con IA que responde en streaming y entiende fotos, y tu GeoCam podrá describir sus fotos con IA. Es la pieza "Asistente IA" del Taller Integrador 2, que entregas con el tag `v2.0.0`.

Vas a poder:

1. Explicar cómo funciona una API de LLM: mensajes, roles, tokens, temperatura y cuotas.
2. Proteger una API key detrás de una Supabase Edge Function.
3. Construir un chat con streaming usando Vercel AI SDK v7.
4. Enviar una foto a un modelo de visión y guardar la respuesta en SQLite, sincronizada con la nube.
5. Decidir cuándo conviene IA en el dispositivo y cuándo en la nube.

Trae listo antes de clase:

- [ ] Tu proyecto de Supabase de la Semana 8 con el login funcionando.
- [ ] Tu GeoCam sincronizada de la Semana 9.
- [ ] Una API key de Gemini creada en Google AI Studio (botón *Get API key*). Guárdala en un lugar seguro: **no** va en el `.env` de la app.
- [ ] Supabase CLI funcionando: `npx supabase --version`.
- [ ] Expo Go actualizado (SDK 57) y el celular en la misma red Wi-Fi que el computador.

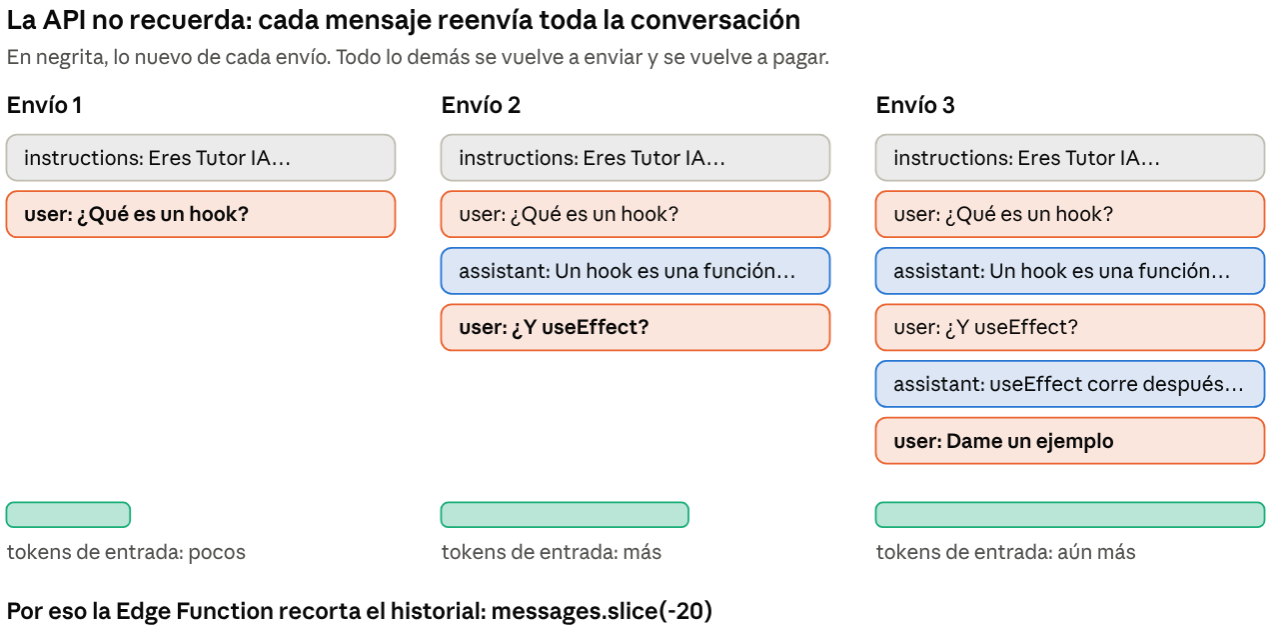
Conceptos

La regla de la semana: tu app nunca habla con Gemini directamente. Habla con un servidor tuyo que guarda la key y decide quién puede usarla.

APIs de LLM

Una llamada a un LLM es una petición HTTP con una lista de mensajes; el modelo responde generando texto token por token.

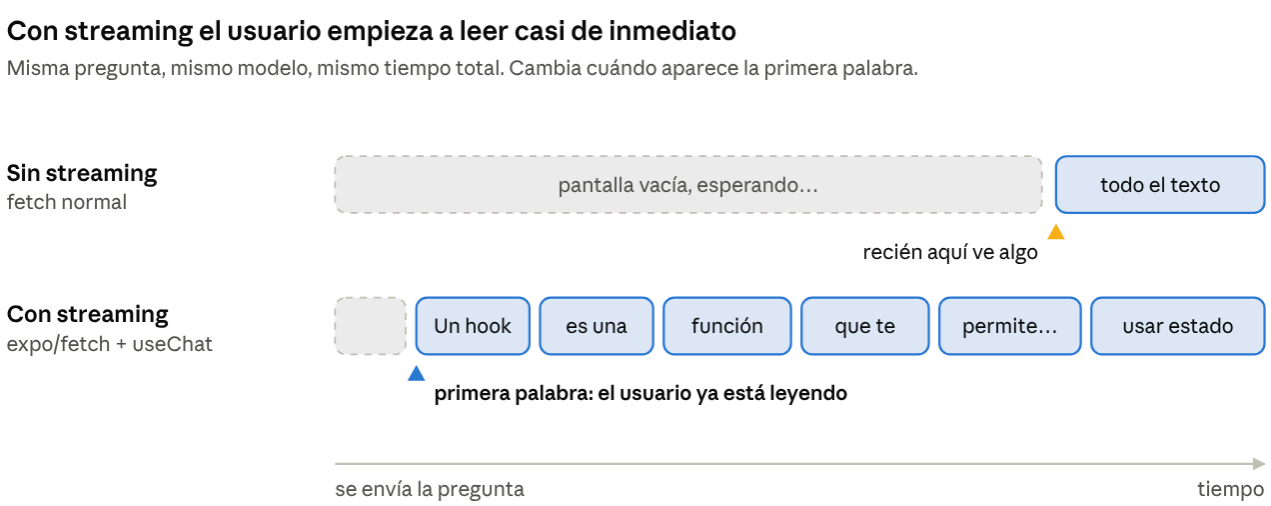
- **Roles:** `instructions` fija cómo se comporta el modelo; `user` es lo que escribe la persona; `assistant`, lo que respondió el modelo.
- **Sin memoria:** la API no recuerda nada. Para que el chat "recuerde", se reenvía el historial en cada mensaje, y por eso las conversaciones largas cuestan más.
- **Tokens:** son trozos de palabra. Se cobra por los de entrada más los de salida.
- **Temperatura:** cerca de 0 da respuestas estables (clasificar, extraer); más alta da variedad.
- **Cuotas:** la capa gratuita limita peticiones por minuto y por día. Al pasarte, la API responde **429** y tu app debe mostrarlo bien.



historial · tres envíos del mismo chat

Streaming

Con streaming el servidor envía la respuesta por partes mientras se genera: ves la primera palabra en menos de un segundo. El `fetch` normal de React Native no lee respuestas por partes; el de `expo/fetch` sí.

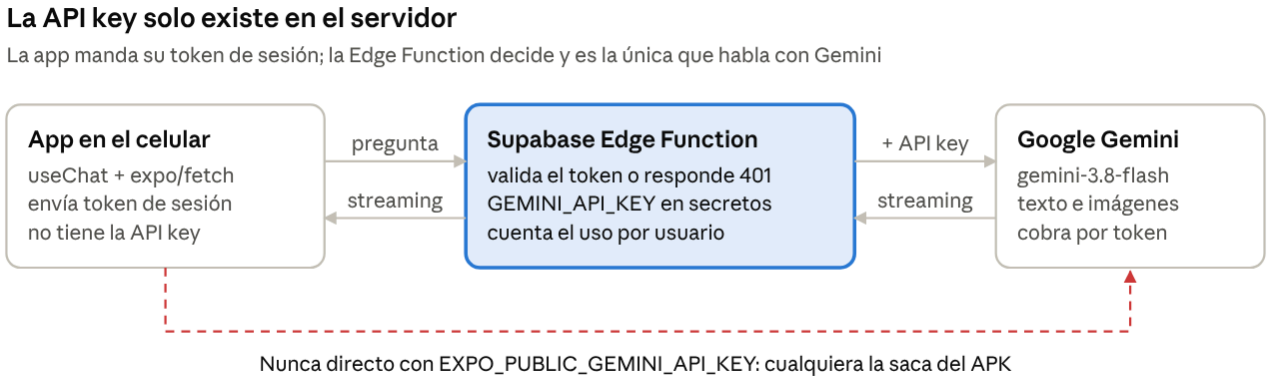


streaming · cuándo aparece la primera palabra

La API key: por qué no va en la app

Toda variable `EXPO_PUBLIC_*` queda escrita en texto plano dentro del bundle de JavaScript. Cualquiera puede descomprimir tu APK, buscar `AIza` y usar tu key.

La solución es un **proxy**: la key vive como secreto de una Supabase Edge Function. Tu app llama a la función con el token de sesión, la función verifica quién eres y solo entonces llama a Gemini.



arquitectura · app, Edge Function y Gemini

La *publishable key* de Supabase sí puede ir en la app porque RLS protege los datos. La *secret key* de Supabase y la key de Gemini, nunca.

Vercel AI SDK v7

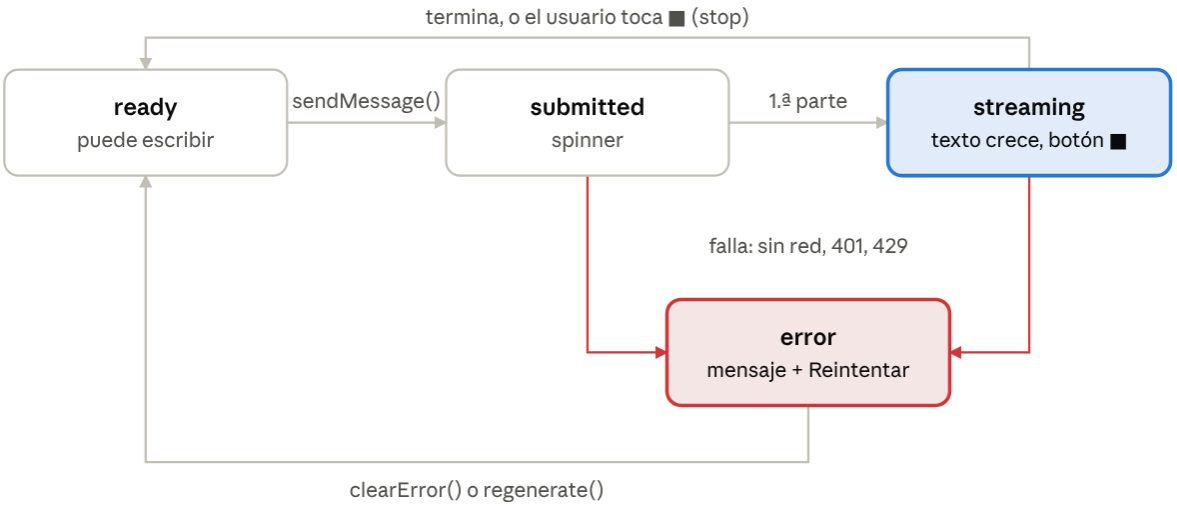
Pieza	Dónde corre	Para qué
`streamText`	Edge Function	Llama al modelo y devuelve el stream

Pieza	Dónde corre	Para qué
`generateText`	Edge Function	Llama al modelo y espera la respuesta completa
`Output.object({ schema })`	Edge Function	Obliga a responder un JSON validado con zod
`useChat`	App	Maneja `messages`, `sendMessage`, `status`, `stop`, `error`
`DefaultChatTransport`	App	URL, headers y `fetch` del chat

Cada mensaje tiene `parts`: partes de texto, archivos (fotos), razonamiento o herramientas. Tu pantalla las recorre y pinta cada una.

useChat pasa por cuatro estados, y la pantalla cambia en cada uno

status es el valor que devuelve useChat. Debajo de cada nombre: qué muestra la UI.



useChat · estados y transiciones

Ojo con los tutoriales viejos: en v7 el prompt de sistema se llama `instructions` (no `system`) y `generateObject` se reemplazó por `generateText` con `output: Output.object(...)`.

Fotos (multimodalidad)

Una foto viaja como una parte más del mensaje: `{ type: 'file', mediaType: 'image/jpeg', url: '...' }`.

- Redúcela antes de enviarla: a 1024 px y calidad 0.7 pesa unos 150 KB en lugar de varios MB.
- Avisa en la pantalla que la foto se envía a Google.

- El modelo puede inventar detalles: muestra su respuesta como sugerencia editable.

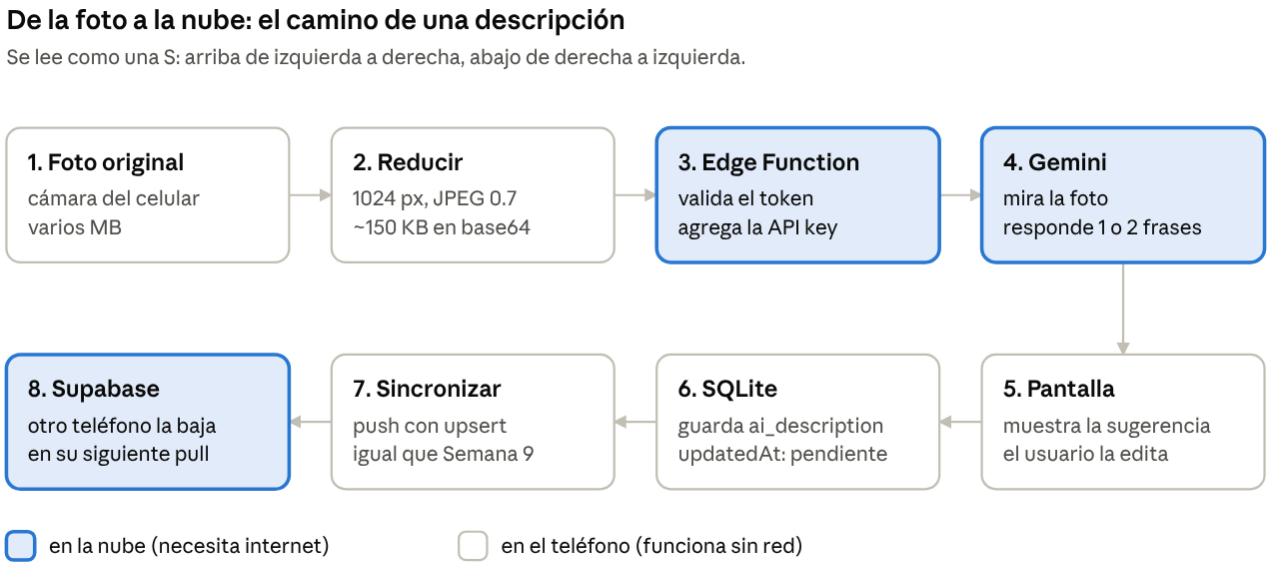


foto · del celular a Gemini y de vuelta a la nube

Edge AI: IA dentro del teléfono

	Nube (Gemini)	Dispositivo (Edge AI)
Sin conexión	No funciona	Funciona
Privacidad	Los datos salen del teléfono	Los datos se quedan
Costo	Por token	Gratis por uso, pero pesa en el APK
Capacidad	Modelos grandes y multimodales	Clasificar, detectar, leer texto
Expo Go	Sí	No: requiere development build

Herramientas para investigar: TensorFlow Lite / LiteRT con `react-native-fast-tflite`, ExecuTorch con `react-native-executorch` y Google ML Kit. Las podrás probar cuando generes builds con EAS en la Semana 14.

Ejemplo guiado: Tutor IA

Construyes un chat que responde en streaming y entiende fotos. La key de Gemini queda en una Edge Function que solo atiende a usuarios con sesión. El proyecto completo está en `s10-tutor-ia.zip`.

1. Crear el proyecto

```
npx create-expo-app@latest s10-tutor-ia --template blank-typescript@sdk-57
cd s10-tutor-ia
npx expo install expo-router expo-constants expo-linking react-native-safe-area-context react-native-screens expo-secure-store expo-image-picker expo-image-manipulator
npm i @supabase/supabase-js ai @ai-sdk/react zod @ungap/structured-clone @stardazed/streams-text-encoding
```

- En `package.json` cambia `"main"` por `"expo-router/entry"` y borra `App.tsx` e `index.ts`.
- Si `npm i` falla con `ERESOLVE` (react-dom), crea un `.npmrc` con `legacy-peer-deps=true`.
- Copia `lib/supabase.ts` y `lib/auth.tsx` de tu Semana 8. Exporta también `SUPABASE_URL` y `SUPABASE_PUBLISHABLE_KEY`.

2. La Edge Function

Crea `supabase/functions/tutor/index.ts`:

```
import { withSupabase } from 'npm:@supabase/server@1';
import { createGoogleGenerativeAI } from 'npm:@ai-sdk/google@4';
import { APICallError, RetryError, convertToModelMessages,
createUIMessageStreamResponse,
  streamText, toUIMessageStream, type UIMessage } from 'npm:ai@7';

const google = createGoogleGenerativeAI({ apiKey:
Deno.env.get('GEMINI_API_KEY') });
const MODELO = Deno.env.get('GEMINI_MODEL') ?? 'gemini-3.8-flash';

export default {
  fetch: withSupabase({ auth: 'user' }, async (req, ctx) => {
    const { messages }: { messages: UIMessage[] } = await req.json();

    const result = streamText({
      model: google(MODELO),
      instructions: `Eres "Tutor IA", tutor de Programación Móvil Avanzada.
Respondes en español, claro y breve.
El estudiante es ${ctx.userClaims?.email}.`,
      messages: await convertToModelMessages(messages.slice(-20)),
      temperature: 0.4,
      maxRetries: 1,
      maxOutputTokens: 1024,
    });
```

```
    return createUIMessageStreamResponse({
      stream: toUIMessageStream({
        stream: result.stream,
        onError: (error) => {
          const real = RetryError.isInstance(error) ? error.lastError :
error;
          if (APICallError.isInstance(real) && real.statusCode === 429) {
            return 'Se agotó la cuota gratuita de Gemini. Espera un minuto.';
          }
          return 'El tutor no pudo responder. Intenta de nuevo.';
        },
      }),
      headers: { 'Content-Type': 'application/octet-stream', 'Content-
Encoding': 'none' },
    });
  }),
};
```

- `withSupabase({ auth: 'user' })`: sin token válido responde 401 y nunca llega a Gemini.
- `messages.slice(-20)`: la API no tiene memoria; limitar el historial cuida tus tokens.
- `onError`: decide qué texto ve el usuario. Nunca le muestres el error crudo de Google.

Despliégala:

```
npx supabase login
npx supabase link --project-ref TU_REF
npx supabase secrets set GEMINI_API_KEY=AIza...
npx supabase functions deploy tutor
```

Sin CLI: Dashboard > Edge Functions > *Deploy a new function* > *Via Editor*. La key se agrega en Edge Functions > *Secrets*.

Comprueba la seguridad: `curl -i -X POST https://TU_REF.supabase.co/functions/v1/tutor -d '{"messages":[]}'` debe responder **401**.

3. Polyfills

El AI SDK necesita `structuredClone`, `TextEncoderStream` y `TextDecoderStream`, que el motor Hermes no trae. Copia `polyfills.ts` del zip e impórtalo como **primera línea** de `app/_layout.tsx`:

```
import '../polyfills';
```

4. El transporte

```
// lib/chatTransport.ts
import { DefaultChatTransport } from 'ai';
import { fetch as expoFetch } from 'expo/fetch';
import { supabase, SUPABASE_PUBLISHABLE_KEY, SUPABASE_URL } from
'./supabase';

export function crearTransporte(funcion = 'tutor') {
  return new DefaultChatTransport({
    api: `${SUPABASE_URL}/functions/v1/${funcion}`,
    fetch: expoFetch as unknown as typeof globalThis.fetch,
    headers: async (): Promise<Record<string, string>> => {
      const { data } = await supabase.auth.getSession();
      const token = data.session?.access_token;
      if (!token) throw new Error('No hay sesión: vuelve a iniciar sesión.');
      return { Authorization: `Bearer ${token}`, apikey:
SUPABASE_PUBLISHABLE_KEY };
    },
  });
}
```

`headers` es una función para que cada mensaje lleve el token vigente. `expo/fetch` es lo que hace posible el streaming.

5. La pantalla de chat

```
const transport = useMemo(() => crearTransporte('tutor'), []);
const { messages, sendMessage, status, error, stop } = useChat({ transport
});
const ocupado = status === 'submitted' || status === 'streaming';

function enviar() {
  sendMessage({ text: texto || '¿Qué ves en esta foto?', files: foto ? [foto]
: undefined });
  setTexto('');
  setFoto(null);
}
```

- Pinta cada mensaje recorriendo `message.parts`: `text` como `<Text>`; `file` de imagen como `<Image>`.
- `status` pasa por `submitted` (esperando la primera palabra), `streaming`, `ready` o `error`.
- Mientras está ocupado, el botón de enviar se vuelve ■ y llama a `stop()`.

Así se ve el Tutor IA y de dónde sale cada parte



maqueta · pantalla del Tutor IA

6. Enviar una foto

```
// lib/foto.ts
export async function reducirFoto(uri: string): Promise<FileUIPart> {
  const ctx = ImageManipulator.manipulate(uri);
  ctx.resize({ width: 1024 });
  const imagen = await ctx.renderAsync();
  const guardada = await imagen.saveAsync({ format: SaveFormat.JPEG,
compress: 0.7, base64: true });
  return { type: 'file', mediaType: 'image/jpeg', filename: 'foto.jpg',
    url: `${guardada.base64}` };
}
```

La foto va dentro del mismo `sendMessage` como parte `file`. La Edge Function no cambia: `convertToModelMessages` la convierte al formato de Gemini.

7. Comprueba

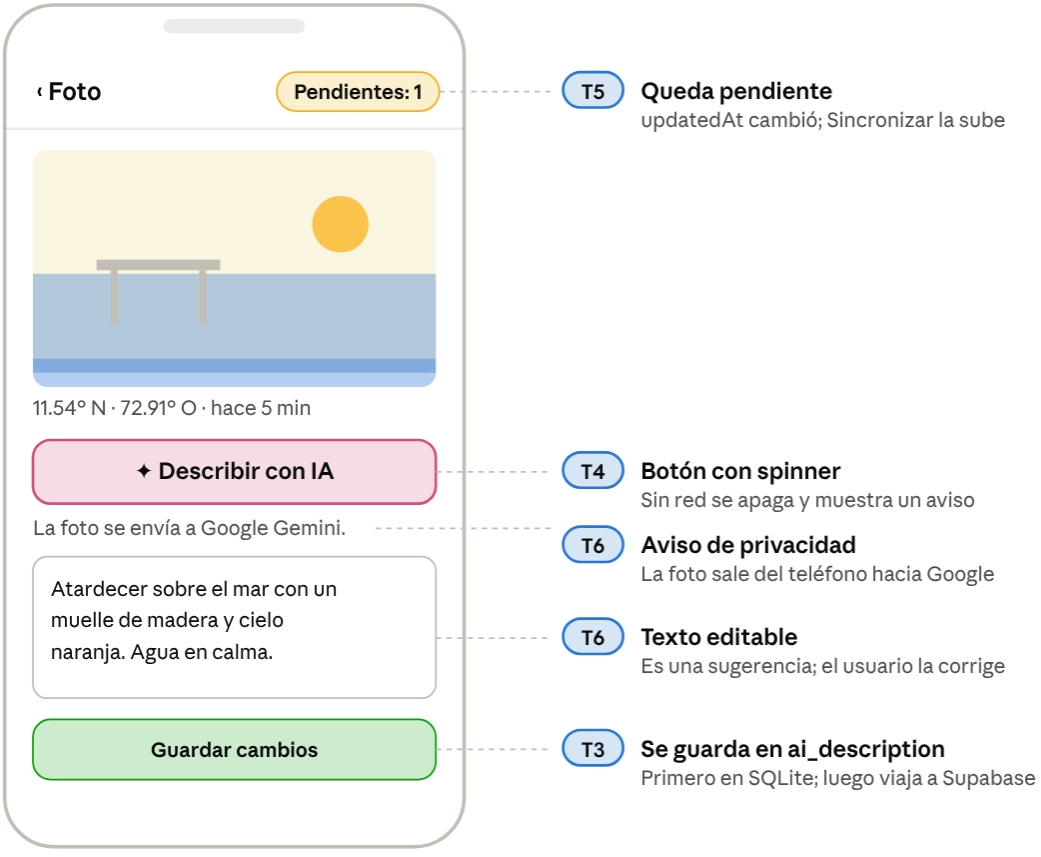
- [ ] Registro e inicio de sesión te llevan al chat.
- [ ] La respuesta aparece por partes, no de golpe.
- [ ] Una foto de código en papel se explica bien.
- [ ] El botón ■ detiene la respuesta.
- [ ] `curl` sin token devuelve 401.
- [ ] `AIza` no aparece en tu repositorio ni en tu `.env`.

Taller en clase: GeoCam ve

Tu GeoCam de la Semana 9 aprende a describir sus fotos. Agregas un botón "✨ Describir con IA"; la descripción se guarda en SQLite y viaja a Supabase con la sincronización que ya tienes.

Esta vez la Edge Function no hace streaming: usa `generateText` y responde un JSON. La llamas con `supabase.functions.invoke`, que envía tu token de sesión automáticamente. Es el patrón para cualquier tarea de IA de una sola respuesta.

GeoCam ve: así queda el detalle de una foto, requisito por requisito



maqueta · detalle de foto en GeoCam ve

Requisitos

#	Requisito	Cómo lo compruebas
T1	Edge Function `describir-foto`: recibe la foto como data URL JPEG y responde `{ descripcion }`. Usa el mismo secreto `GEMINI_API_KEY`	`curl` sin token responde 401
T2	`services/vision.ts` reduce la foto a 1024 px JPEG 0.7 y llama la función con `supabase.functions.invoke`	La petición pesa menos de 300 KB
T3	Columna `ai_description` en SQLite (migración nueva, sin borrar `drizzle/`) y en Supabase	Tus fotos anteriores siguen en la app
T4	Botón en el detalle de la foto, con spinner; apagado y con aviso cuando no hay red	Pruébalo en modo avión
T5	Al guardar, `updatedAt` cambia y la foto queda pendiente; al sincronizar sube; en otro teléfono baja	Mira la columna en Supabase Table Editor
T6	La descripción se puede editar (es una sugerencia) y la pantalla avisa que la foto se envía a Google	Edita sin red y sincroniza después

Pistas

- La función se parece a `tutor`, pero con `generateText`. La imagen va así: `{ type: 'file', mediaType: 'image/jpeg', data: <base64 sin el prefijo data:> }`.
- Si la función responde 4xx o 5xx, `invoke` devuelve un `error` de tipo `FunctionsHttpError`. Tu mensaje está en `await error.context.json()`.
- En SQL: `alter table photos add column ai_description text;`. En Drizzle: `aiDescription: text('ai_description')` y luego `npm run db:generate`. Revisa que el `.sql` nuevo sea un `ALTER TABLE`.
- En el repositorio, `setDescription` debe cambiar también `updatedAt`. Sin eso, la descripción nunca se sube.
- En `services/sync.ts` agrega el campo en **los dos** mapeos: el de push y el de pull.

Si terminas antes: muestra las primeras palabras de la descripción en la lista de fotos, o agrega "Describir todas" con una pausa de 5 segundos entre fotos para no agotar la cuota.

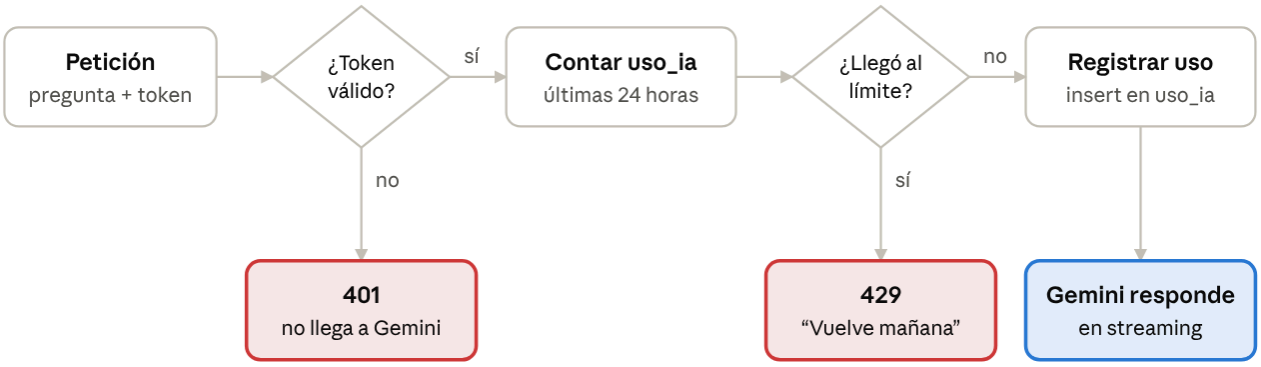
Trabajo para la casa: C1–C6

Con estos seis retos tu asistente IA pasa de demo a pieza de producto. Puedes aplicarlos al Tutor IA o directamente a la app de tu Taller Integrador 2. Entregas el repositorio con el tag `v2.0.0` y `AI-LOG.md` actualizado.

Reto	Qué se pide	Pista
C1 · Historial persistente	La conversación se guarda en SQLite con Drizzle; al abrir la app sigue donde iba. Botón "Nueva conversación"	`useChat({ messages: inicial, onFinish: ({ messages }) => guardar(messages) })` con upsert por `id`
C2 · Instrucciones con tus datos	Pantalla de perfil (nombre, tema, nivel). El servidor arma `instructions` con esos datos y los valida con zod	`DefaultChatTransport({ body: async () => ({ perfil }) })`; en el servidor, `.catch()` para valores inválidos
C3 · Salida estructurada	De 3 a 5 tarjetas de repaso (pregunta y respuesta) generadas a partir de tu conversación	Nueva función con `generateText` + `output: Output.object({ schema })`
C4 · Manejo de errores	Banner sin conexión, envío apagado sin red, mensajes claros para 401, 429 y falla de red, botón Reintentar	`useNetworkState` de expo-network y `regenerate()`. Si el token venció, cierra la sesión
C5 · Límite por usuario	Máximo N peticiones de IA cada 24 horas por usuario, contadas en Supabase	Tabla `uso_ia` con RLS (ver e insertar lo propio, sin borrar). Cuenta con el cliente del usuario, no con la secret key
C6 · Bitácora y pruebas	`AI-LOG.md` con al menos 2 errores reales de la IA y `PRUEBAS.md` con tu matriz de escenarios	Incluye modo avión, límite, sesión vencida y buscar `AIza` en el bundle

C5: cada petición pasa dos controles antes de llegar a Gemini

Todo ocurre dentro de la Edge Function; la app solo ve la respuesta final.



La cuenta usa el cliente del usuario: RLS le deja ver e insertar solo sus filas, y nunca borrarlas.

C5 · límite diario por usuario

Para probar C5 sin gastar cuota: `npx supabase secrets set LIMITE_DIARIO=3` y haz cuatro preguntas.

Entrega

- [ ] Repositorio público con el tag `v2.0.0` (`git tag v2.0.0 && git push --tags`).
- [ ] `.env.example` en el repo; `.env` y cualquier key, **fuera**.
- [ ] Carpeta `supabase/functions/` y los `.sql` que ejecutaste.
- [ ] `README.md` con capturas del chat, la foto descrita y el mensaje de límite.
- [ ] `AI-LOG.md` y `PRUEBAS.md` completos.

Debes poder explicar

- ¿Dónde está tu key de Gemini y cómo lo demuestras?
- ¿Qué pasa si alguien envía un perfil falso o un `tema` inventado?
- ¿Por qué el límite se cuenta con el cliente del usuario y no con la secret key?
- ¿Qué se guarda en tu teléfono y qué se envía a Google?

Errores frecuentes y documentación

Casi todo falla por tres cosas: el polyfill, el `fetch` equivocado o el secreto mal puesto.

Síntoma	Causa	Solución
`Property 'structuredClone' doesn't exist`	Falta el polyfill o no es la primera línea	`import '../polyfills'` arriba de todo en `app/_layout.tsx`
La respuesta llega toda de golpe	Usaste el `fetch` normal	`fetch` de `expo/fetch` en el transporte
`ERESOLVE could not resolve` (react-dom)	Conflicto de dependencias	`.npmrc` con `legacy-peer-deps=true`
401 `Unauthorized`	Sin sesión o token vencido	`headers: async () => ...` y vuelve a iniciar sesión
500 y en los logs `API key not valid`	Secreto mal escrito o no desplegado	`npx supabase secrets list`, `secrets set` y vuelve a hacer `deploy`
429 "cuota"	Límite de la capa gratuita de Gemini	Espera un minuto o usa `secrets set GEMINI_MODEL=gemini-3.5-flash-lite`
`models/... is not found`	Modelo retirado (los Gemini 2.5 se apagaron en octubre de 2026)	Usa `gemini-3.8-flash` o `gemini-3.5-flash-lite`
La IA te sugiere `EXPO_PUBLIC_GEMINI_API_KEY`	Tutoriales viejos	No lo hagas; anótalo en tu AI-LOG
La IA usa `system:` o `generateObject`	Ejemplos de AI SDK v4/v5	En v7: `instructions:` y `generateText` con `Output.object`
Error al preparar la base tras agregar `ai_description`	Borraste o regeneraste `drizzle/`	Solo `npm run db:generate`; nunca borres migraciones viejas

Los logs de tus funciones están en Dashboard > Edge Functions > (función) > Logs.

Documentación

- AI SDK · Expo Quickstart
- AI SDK · useChat
- AI SDK · Proveedor Google

- Supabase · Securing Edge Functions
- Supabase · @supabase/server
- Google AI Studio
- Expo · ImageManipulator
- Edge AI: react-native-executorch y react-native-fast-tflite